

LeanSearch v2: Global Premise Retrieval for Lean 4 Theorem Proving

Guoxiong Gao^{1,2} Zeming Sun^{1,3} Jiedong Jiang⁴ Yutong Wang^{1,2}
Jingda Xu² Peihao Wu² Bryan Dai^{2,†} Bin Dong^{5,6,7,8,†}

¹School of Mathematical Sciences, Peking University

²IQuest Research

³Research Institute for Mathematical Sciences, Kyoto University

⁴Westlake Institute for Advanced Study, Westlake University

⁵Beijing International Center for Mathematical Research and the New Cornerstone Science Laboratory, Peking University

⁶Center for Machine Learning Research, Peking University

⁷Center for Intelligent Computing, Great Bay Institute for Advanced Study, Great Bay University

⁸Zhongguancun Academy

[†]Corresponding authors: cbdai@iquestlab.com, dongbin@math.pku.edu.cn

Abstract

Proving theorems in Lean 4 often requires identifying a scattered set of library lemmas whose joint use enables a concise proof—a task we call *global premise retrieval*. Existing tools address adjacent problems: semantic search engines find individual declarations matching a query, while premise-selection systems predict useful lemmas one tactic step at a time. Neither recovers the full premise set an entire theorem requires. We present LeanSearch v2, a two-mode retrieval system for this task. Its standard mode applies a hierarchy-informalized Mathlib corpus with an embedding–reranker pipeline, achieving state-of-the-art single-query retrieval without domain-specific fine-tuning (nDCG@10 of 0.62 vs. 0.53 for the next-best system). Its reasoning mode builds on standard mode as its retrieval substrate, targeting global premise retrieval through iterative sketch-retrieve-reflect cycles. On a 69-query benchmark of research-level Mathlib theorems, reasoning mode recovers 46.1% of ground-truth premise groups within 10 retrieved candidates, outperforming strong reasoning retrieval systems (38.0%) and premise-selection baselines (9.3%) on the same benchmark. In a controlled downstream evaluation with a fixed prover loop, replacing alternative retrievers with LeanSearch v2 yields the highest proof success (20% vs. 16% for the next-best system and 4% without retrieval), confirming that retrieval quality propagates to proof generation. We have open-sourced all code, data, and benchmarks.¹²

1 Introduction

Rapid advances in Lean 4 and its central library Mathlib [The mathlib Community, 2020] have made formalization an increasingly accepted practice in the mathematical community, with the library now containing over one hundred thousand verified declarations spanning algebra, analysis, topology, and number theory. AI systems have further accelerated this trend, demonstrating growing capability in generating formal proofs [Hubert et al., 2025, Ren et al., 2025, Achim et al., 2025]. A central challenge in working within this ecosystem is leveraging what Mathlib already contains. Locating the right library lemma carries two complementary benefits: a well-chosen result can collapse a lengthy derivation into a single tactic invocation, and reusing existing results is

¹Code and data are available at <https://github.com/frenzymath/LeanSearch-v2>.

²The standard mode is publicly available with API access at <https://leansearch.net/>.

the community’s explicit coding standard. Mathlib prioritizes generality and actively discourages code duplication [Mathlib Community, 2024]; indeed, “being able to find the library theorems you need constitutes an important part of formalization” [Avigad and Massot, 2020]. This norm has direct consequences for automated provers: without effective retrieval, an AI system must repeatedly prove local versions of lemmas that already exist in the library for each problem it encounters, wasting computation and increasing proof difficulty. Yet finding the right lemmas is hard, not primarily because the library is large, though its scale compounds the problem, but because the connections require mathematical reasoning. Effectively applying a powerful theorem often demands reshaping the proof state to match its interface, generalizing or transforming the current goal in ways that temporarily complicate it, only to yield a far simpler overall proof by routing through a result in a seemingly unrelated area. The relevant lemmas are linked to the problem not by shared vocabulary but by the logical architecture of a proof strategy. Retrieving the *set* of supporting lemmas on which a complete proof should rest is a task we term *global premise retrieval*: identifying which library results are essential to the theorem’s overall proof strategy, considering the needs of the entire proof rather than what simplifies any single intermediate state.

Example: Global Premise Retrieval

Problem. Prove that $\frac{x^p-1}{x-1}$ is irreducible in $\mathbb{Z}[x]$ for every prime p .

Supporting premises (from 3 distant Mathlib modules):

Lemma	Statement / Role
<code>geom_sum_mul</code> Algebra.Ring.GeoSum	$(\sum_{i=0}^{n-1} x^i)(x-1) = x^n - 1$ Establishes the algebraic identity.
<code>Polynomial.cyclotomic_prime</code> RingTheory.Polynomial.Cyclotomic.Basic	$\Phi_p(x) = \sum_{i=0}^{p-1} x^i$ Identifies the quotient as the p -th cyclotomic polynomial.
<code>Polynomial.cyclotomic.irreducible</code> RingTheory.Polynomial.Cyclotomic.Roots	Φ_n is irreducible over $\mathbb{Z}$ for $n > 0$. Concludes the proof in one step.

Note. The problem statement shares no vocabulary with these premises (“cyclotomic,” “geometric sum,” “ Φ ” all absent). Identifying them requires reasoning about proof strategy, not lexical matching; the three lemmas chain together to close the proof, and missing any one leaves the argument incomplete.

Figure 1: A global premise retrieval instance requiring three lemmas from distant Mathlib modules.

Figure 1 illustrates why this task is difficult. The classical proof of irreducibility of $(x^p-1)/(x-1)$ proceeds by substitution and Eisenstein’s criterion, but a more efficient proof in Lean bypasses this re-derivation entirely by routing through existing library results on cyclotomic polynomials, resolving the goal in one step once the algebraic equivalence is established. Recognizing that such a result exists is the first challenge, since the problem statement shares no vocabulary with the cyclotomic machinery. The second challenge is pinpointing the three specific lemmas that chain together to close the proof; missing any one leaves the argument incomplete. In graduate-level mathematics and beyond, this pattern is extremely common: a proof relies on a handful of supporting results dispersed across the library, connected not by shared terminology but by the logical architecture of the proof strategy.

Two active research lines build retrieval tools for mathematics, yet neither addresses global premise retrieval. The *premise selection* line [Yang et al., 2023, Tao et al., 2025, Shen et al., 2025, Zhu et al., 2025] retrieves premises for a given proof state, improving tactic-level proving but operating at per-state granularity without considering what an entire proof requires. The *reasoning retrieval* line, catalyzed by the BRIGHT benchmark [Su et al., 2024], advances retrieval for queries requiring multi-step inference through reasoning-aware encoders, multi-stage pipelines, and LLM-based reranking [Shao et al., 2025, Long et al., 2025b, Yao et al., 2025]. However, current benchmarks evaluate each retrieved document’s relevance independently; these systems have not yet addressed the regime where multiple premises from different mathematical areas must jointly support a proof, nor the formal-language setting where problem statements may be deliberately reshaped to match a library theorem’s interface. Neither line is designed to recover a *logically coherent set* of premises for an entire theorem, where the elements are connected by the internal logic of a proof strategy rather than merely co-relevant to a query.

We present **LeanSearch v2**, a retrieval system designed for this task. Its *standard mode* serves as the foundation: it builds a hierarchy-informalized Mathlib corpus and retrieves against it with a two-stage embedding–reranker pipeline, without domain-specific fine-tuning. Its *reasoning mode* builds on standard mode as its retrieval substrate to target global premise retrieval directly: given a theorem statement, it generates a proof sketch, retrieves candidates for each proof step via standard mode, filters and judges the results, and iteratively revises the sketch until a coherent supporting-lemma set emerges. We evaluate on two new benchmarks: **MathlibQR** (Mathlib Queries), a 200-declaration, 946-query evaluation for standard-mode search, and **MathlibMPR** (Mathlib Merged Pull Requests), a 69-theorem benchmark with expert-annotated premise groups (one to eight per theorem) for global premise retrieval. On MathlibQR, standard mode achieves nDCG@10 of 0.62 against 0.53 for the next-best system. On MathlibMPR, reasoning mode recovers 46.1% of ground-truth premise groups within 10 retrieved candidates, outperforming both reasoning retrievers (INF-X [Yao et al., 2025], ReasonIR [Shao et al., 2025], DIVER [Long et al., 2025b]) and premise-selection baselines (ReProver [Yang et al., 2023], LeanStateSearch [Tao et al., 2025], LeanPremise [Zhu et al., 2025]) on the same benchmark. A controlled downstream experiment further confirms that this retrieval advantage propagates to proof success when the prover loop is held fixed.

Although we evaluate the downstream effect of plugging our retriever into a prover system (§4.3), we deliberately position this work in the retrieval-research stratum rather than the proof-generation stratum. Strong agent systems [Chen et al., 2025, Liu et al., 2026, Achim et al., 2025, Ju et al., 2026] have shown a capable agent can compensate for a weaker retriever through repeated query reformulations, or bypass retrieval entirely by proving needed lemmas locally. However, closing the gap between a proof that merely compiles and an efficient, idiomatic proof still depends on identifying and using existing results—a core challenge that makes global premise retrieval a meaningful task in its own right.

Contributions.

- We formulate *global premise retrieval*, the task of recovering the logically coherent set of library lemmas that an entire theorem’s proof requires, and construct two expert-curated Lean 4 benchmarks: MathlibQR (200 declarations, 946 queries) for single-query search, and MathlibMPR (69 theorems with structured premise-group annotations and alternative proof routings) for set-valued premise retrieval.
- We present LeanSearch v2, a staged retrieval system whose standard mode achieves state-of-the-art Mathlib search without domain-specific fine-tuning, and whose reasoning mode builds on it through iterative sketch-retrieve-reflect cycles to target global premise retrieval directly, outperforming both reasoning-intensive retrievers and premise-selection baselines on our benchmarks. A controlled downstream experiment further confirms that retrieval quality propagates to end-to-end proof success.
- To our knowledge, this is the first work to bring reasoning-retrieval techniques to the Lean premise retrieval problem.

2 Related Work

2.1 Reasoning Retrieval

Recent benchmarks formalize retrieval tasks that require reasoning beyond lexical matching [Welleck et al., 2021, Su et al., 2024, Ju and Dong, 2025, Killingback and Zamani, 2025, Abdallah et al., 2026]. Welleck et al. [2021] introduced mathematical reference retrieval in informal text. BRIGHT [Su et al., 2024] subsequently defined *reasoning-intensive retrieval*, revealing a large accuracy drop when standard dense retrievers face queries requiring multi-step inference. Follow-up methods address this through reasoning-aware encoders [Shao et al., 2025, Das et al., 2025], multi-stage pipelines [Long et al., 2025b], and LLM-based reranking [Weller et al., 2025, Long et al., 2025a]. These benchmarks and methods target informal-text corpora such as web documents and scientific articles, evaluating each retrieved document’s relevance independently.

2.2 Retrieval on Mathlib

Semantic search engines. Several tools support natural-language search over Lean 4’s Mathlib. Moog³ and Loog⁴ provide semantic and syntactic search respectively. LeanSearch [Gao et al., 2024a] applies off-the-shelf embeddings to a bilingual formal–informal corpus. LeanExplore [Asher, 2025] combines dense, sparse, and graph-based signals. Lean Finder [Lu et al., 2025] fine-tunes on synthesized user-intent queries. Each system returns a ranked list for a single natural-language query.

Premise selection. A separate line retrieves premises conditioned on a local proof state. Re-Proof [Yang et al., 2023] introduced dense retrieval over state–premise pairs in Lean. Subsequent work refines the paradigm with improved negative mining [Tao et al., 2025], integration into end-to-end provers [Shen et al., 2025], automated reasoning backends [Zhu et al., 2025], and graph-augmented embeddings [Petrovčič et al., 2025]. These systems focus on finding premises that advance the current local proof state.

3 Methodology

LeanSearch v2 operates in two modes. *Standard mode* (§3.1) retrieves individual Mathlib declarations from a natural-language query. It serves as both a standalone search engine and the retrieval substrate for the second mode. *Reasoning mode* (§3.2) targets global premise retrieval: given a formal theorem statement, it iteratively sketches proof strategies and queries standard mode, assembling the set of library premises needed for a concise proof.

3.1 Standard mode

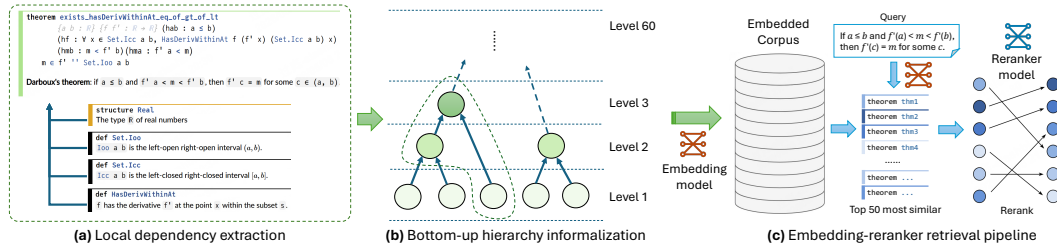


Figure 2: Standard mode pipeline. (a) Jixia extracts each declaration and its dependencies from Mathlib. (b) Declarations are informalized bottom-up so that each description can draw on already-informalized dependencies. (c) Queries and corpus items are embedded for cosine-similarity retrieval; a reranker refines the top candidates.

Standard mode serves as both a standalone search engine and the retrieval substrate for reasoning mode (Figure 2). It comprises two components: an informalized Mathlib corpus and an embedding–reranker retrieval pipeline.

Informalized corpus preparation. The retrieval corpus covers all extractable declarations from Mathlib v4.28.0-rc1 (definitions, theorems, type classes, and other named constants), each paired with a concise natural-language description of its mathematical meaning.

We extract declarations using Jixia,⁵ a publicly available, open-source, non-intrusive static analysis tool for Lean 4 (Figure 2a). Its Declaration plugin provides source-level metadata (kind, arguments, return type, source range); its Symbol plugin records elaborated constants, their types, and the inter-declaration reference graph. To improve extraction coverage, Jixia is extended to (i) extract hidden compiler-generated declarations such as auxiliary definitions produced for structures (Figure 3), (ii) recover declarations from elaborated terms rather than surface source alone, and (iii) support customized Lean runtime options so that extracted representations match the configuration used during corpus construction.

³<https://www.moogole.ai>

⁴<https://loogle.lean-lang.org/>

⁵<https://github.com/frenzymath/jixia>

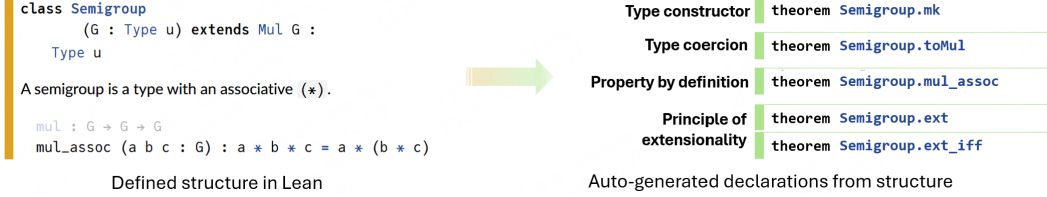


Figure 3: Hidden auto-generated declarations in Mathlib.

The reference graph induces a directed acyclic graph (DAG) over declarations. We topologically sort it and informalize bottom-up (Figure 2b): each call receives the already-informalized descriptions of the target’s dependencies as context, so that descriptions are grounded in mathematical concepts rather than opaque formal identifiers; this dependency-aware informalization strategy is known to produce more grounded descriptions than flat, context-free informalization [Gao et al., 2024b]. Informalization uses Qwen3-32B [Yang et al., 2025] with task-specific prompts; failures are reprocessed with Gemini 2.5 Pro [Comanici et al., 2025] as a fallback. Each corpus item stores declaration kind, fully qualified name, type signature, value (when available), source location, dependencies, and the informalized description.

Embedding and reranker pipeline. Each corpus item is encoded into a fixed-dimensional vector using Qwen3-Embedding-8B [Zhang et al., 2025], a general-purpose embedding model that ranks among the strongest on MTEB [Muennighoff et al., 2023], the standard benchmark suite for text embedding quality, without domain-specific fine-tuning (Figure 2c). For each item we carefully compose a structured passage from a tuned template that combines a task instruction with well-formatted formal and informal information, including the declaration kind, type signature, informalized description, and, for definitions, classes, and instances, the value field, whose content carries essential semantic meaning (unlike theorem values, which are proof terms). The passages are embedded and indexed for cosine-similarity retrieval.

The top-50 candidates are reranked by Qwen3-Reranker-8B [Zhang et al., 2025], which scores each query-candidate pair by predicting a binary relevance token and using $P(\text{yes})$ as the score. The reranker receives a kind-aware instruction with special handling for definition-type declarations, aiming to compensate for the retrieval difficulty on definitions that prior approaches [Gao et al., 2024a], which did not distinguish declaration kinds, consistently exhibited. An ablation showing the impact of this presentation design on definition retrieval is in Appendix D.1. The reranked list constitutes standard mode’s final output.

No task-specific training is performed: neither the embedder nor the reranker is fine-tuned on Mathlib data. Domain knowledge enters through informalization and kind-aware prompting, keeping the system straightforward to maintain as Mathlib evolves, and to transfer to other rapidly growing AI-generated formal libraries whose style and conventions may differ substantially from Mathlib.

3.2 Reasoning mode

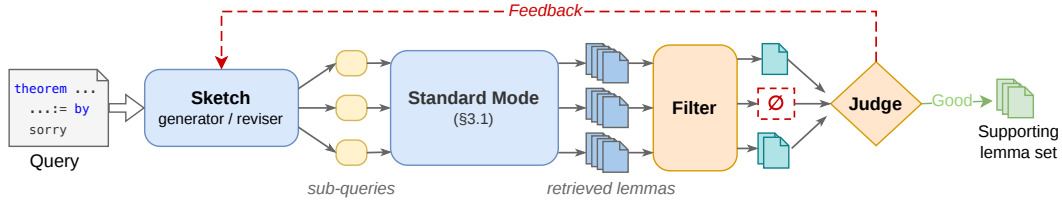


Figure 4: Reasoning mode pipeline. The sketch generator decomposes a target theorem into sub-queries. Each sub-query is sent to standard mode (§3.1), and a filter vets the retrieved candidates, potentially returning an empty set ($\emptyset$). The judge inspects all filtered outputs and either accepts the supporting lemma set or sends structured feedback (red dashed arrow) to the sketch reviser for another iteration.

Reasoning mode targets global premise retrieval through an iterative sketch-retrieve-reflect loop built on top of standard mode (Figure 4). Given a target theorem, the loop decomposes it into sub-queries via a sketch generator, retrieves candidates for each sub-query through standard mode (§3.1),

filters the results, and submits the filtered map to a feasibility judge that either accepts the supporting lemma set or routes structured feedback back to a sketch reviser for another iteration. Standard mode is treated as a black-box retrieval service: reasoning mode only issues natural-language queries against it and consumes ranked results, never reaching into the embedder or reranker. This separation lets the loop reason about *retrieval signals*, including the absence of useful results for a given query, rather than about embeddings or scores, and keeps each mode independently improvable. We use Claude Sonnet 4.5 [Anthropic, 2025] for sketch generation, judge, and revision, and Kimi K2 Instruct [Team et al., 2025] for the filter.

Sketch generation and revision. Given a target theorem (its formal statement and an informal description), the sketch generator proposes a step-by-step proof outline (Figure 4, left). Each step contains a natural-language description of the mathematical move, any auxiliary context it relies on, and a retrieval query phrased in the style that standard mode expects. The generator is not asked to write Lean code or name specific Mathlib lemmas; its task is to commit to a proof strategy at the granularity of *which results should exist and how each contributes*, leaving lemma identification to the retrieval substrate. The sketch reviser shares this interface but additionally consumes the previously rejected sketch and the judge’s feedback.

Document filter. For each sub-query in the current sketch, standard mode returns a ranked list of candidate declarations (Figure 4, center). The filter LLM inspects each candidate’s metadata (kind, type signature, value, informalized statement) alongside the query and independently labels it as relevant or not. Crucially, the filter may return an empty set: when no candidate is genuinely useful for the planned step, none is kept. This empty signal allows the judge to distinguish “the retriever found support for this step” from “the retriever found nothing useful here,” two situations that demand different revisions and that a top- k rule would conflate.

Feasibility judge and reflection loop. After all per-step filtered outputs are collected, the judge inspects the full filtered map and decides whether the sketch as a whole is supportable by the library (Figure 4, right). Judgment is binary: if supportable, the loop terminates and the per-query filtered lists are pooled with an nDCG-style rank discount to form reasoning mode’s final output (formula and deduplication rule in Appendix B.1); if unsupportable, the judge produces structured feedback identifying which steps failed and why (e.g., a missing premise, a candidate suggesting a different proof route, or an unsatisfiable type constraint), and routes control to the sketch reviser for a revised sketch that re-enters the retrieve-filter-judge pipeline. We cap the loop at three rounds of revision past the initial sketch; if the judge has not accepted by then, we take the latest filtered output regardless. An ablation against a no-reflection variant of the loop is in Appendix D.3. Loop-iteration statistics are reported in Appendix C.1.

4 Experiments

We evaluate LeanSearch v2 across three tasks. **Search** (§4.1) evaluates standard mode (§3.1) in the canonical Mathlib search setting: given a natural-language query, retrieve the matching declaration, scored by nDCG@ k and Recall@ k . **Global premise retrieval** (§4.2) evaluates reasoning mode (§3.2) on the task formulated in §1: given a theorem statement, retrieve the set of supporting lemmas. Benchmark construction and ground-truth annotation are detailed in §4.2. **Prove** (§4.3) measures retrieval quality extrinsically: each retriever is plugged into a fixed proof-generation workflow (the *simple reflection loop*, see §4.3), and we report end-to-end proof success on FATE-H [Jiang et al., 2025], a 100-problem benchmark sitting at the graduate-level tier of the FATE algebra series (FATE-M / H / X span undergraduate to post-PhD difficulty), and on MathlibMPR-Prop, a 50-problem subset of MathlibMPR (§4.2). The first task compares standard mode against existing Mathlib search engines, while the latter two compare reasoning mode against both reasoning-intensive retrievers and Lean-side premise-selection systems; to our knowledge no prior evaluation places these two families on a single Lean benchmark.

4.1 Search: query-to-declaration retrieval

The Search task measures standard mode’s quality as a Mathlib search engine: given a natural-language query, recover the single matching declaration. We evaluate on **MathlibQR** (Mathlib

Table 1: **Search task on MathlibQR**. Numbers are nDCG@ k and Recall@ k over 810 query rows on the shared 171-declaration corpus subset (intersection of the four systems’ Mathlib snapshots). *LLM judge* reports mean rank ($\downarrow$ better, range 1–4) by Claude Sonnet 4.5 over 2430 blinded judgments per system. Best per column in **bold**.

System	nDCG@ k			Recall@ k			LLM judge
	1	5	10	10	50	100	mean rank
LeanExplore	0.246	0.358	0.393	0.569	0.743	0.789	3.32
LeanFinder	0.370	0.514	0.533	0.698	0.824	0.875	2.87
LeanSearch v2 (retriever-only)	0.340	0.472	0.494	0.657	0.790	0.830	2.18
LeanSearch v2 (rerank)	0.470	0.601	0.623	0.780	0.847	0.858	1.63

Queries), a benchmark we construct by sampling 200 Mathlib declarations across all major mathematical subjects and all major Mathlib declaration kinds, with up to six query styles per declaration (Lean-flavored, LaTeX, plain English, conceptual slogan, informal nickname, and special-case instance) for **946** query rows in total; ground-truth declarations are tagged *Easy* or *Hard* based on mathematical obscurity and technical complexity (rubric in Appendix A.1); every query inherits its declaration’s tag. The benchmark intentionally spans diverse query styles and declaration kinds so that no system can succeed by overfitting to a single style; further examples and full curation details are provided in Appendix A.1.

We compare four systems: LeanExplore [Asher, 2025], a hybrid dense / sparse / graph engine; LeanFinder [Lu et al., 2025], a fine-tuned dense retriever with a learned reranker; LeanSearch v2 with its full retrieve-rerank pipeline; and a retriever-only ablation of LeanSearch v2 that drops the Qwen3-Reranker-8B stage, which we report alongside the full system to expose the reranker’s marginal contribution. Because these systems target different Mathlib snapshots (i.e., a declaration available to one may be absent in another), the main report uses the *fair* subset of MathlibQR, the 810 query rows whose ground-truth declarations exist in every system’s snapshot, so that the comparison controls for snapshot drift rather than retrieval quality; results on looser subsets (937-row and full 946-row) are in Appendix C.4.

We report nDCG@ k and Recall@ k with binary single-document relevance, and additionally an LLM-as-judge protocol following Asher [2025]: each system’s top-5 returns are presented to Claude Sonnet 4.5 in three independent random permutations per query, and the judge produces a strict total ordering 1–4 across the four systems. We report mean rank (lower is better) over 2430 judgments per system; position-bias and cross-round agreement diagnostics, including a primacy-bias correction, are reported in Appendix B.2.

LeanSearch v2 (Table 1, last row) leads on most reported metrics. The LLM judge column corroborates this ranking, with LeanSearch v2 attaining a mean rank of 1.63. Appendices C.2 and D.1 report per-slice breakdowns and a step-wise ablation: v2-rerank leads on nearly every per-slice cell, with the rerank step accounting for roughly +10 nDCG/Recall points at $k \leq 10$ and kind-aware prompting adding another +2–3.

4.2 Global premise retrieval

The global premise retrieval task evaluates reasoning mode (§3.2) on the formulation of §1: given a theorem statement, recover the full set of library lemmas its proof requires. We assemble **MathlibMPR** (Mathlib Merged Pull Requests), a benchmark of 69 theorems sourced from merged Mathlib pull requests. For each PR we extract its main statement, an informal description of the goal, and the set of premises actually invoked by its tactic proof. Ground truth is structured into *premise groups* (one to eight per query, mean 2.96), where lemmas within a group are interchangeable for a given proof step; for a subset of queries, formalization experts additionally annotate equivalent proof routings as *alternative routings*. The benchmark’s central design choice is mathematical substance: candidate PRs are filtered to retain only theorems that are not trivial restatements, special cases, or notational rewrites, so that even single-group queries reflect nontrivial use of a premise. Compared to traditional reasoning retrieval benchmarks, global premise retrieval poses significantly greater difficulty and exhibits distinct characteristics; a detailed comparison is given in Appendix A.2.3. Full curation details and examples are provided in Appendix A.2.

Table 2: **Global premise retrieval task on MathlibMPR**. Numbers are percentages. *Recall (group)*: fraction of ground-truth premise groups hit by the top- k , macro-averaged. *Covered*: 1 iff some complete proof routing (original or expert-annotated alternative) has all of its premise groups hit by the top- k . Top block: reasoning retrievers consuming informal description + formal statement; sub-rows give published ablations. Bottom block: premise-selection systems consuming the extracted proof state. Best per column in **bold**.

System	Recall@ k (group)					Covered@ k				
	5	10	20	30	50	5	10	20	30	50
<i>Reasoning retrieval (informal + formal query)</i>										
INF-X-Retriever	21.2	28.2	33.9	40.0	42.6	15.9	18.8	21.7	24.6	24.6
+ query rewriter	13.2	17.4	25.2	28.4	38.3	10.1	10.1	17.4	17.4	24.6
ReasonIR	23.9	26.9	33.3	35.6	43.8	17.4	18.8	20.3	21.7	30.4
DIVER (full pipeline)	27.7	38.0	46.6	52.8	55.0	18.8	24.6	29.0	33.3	37.7
retriever only	21.6	24.7	40.6	45.2	48.5	15.9	15.9	26.1	31.9	33.3
retriever + query expansion	22.2	27.6	40.8	43.0	47.9	15.9	17.4	26.1	27.5	31.9
retriever + GroupRank	29.7	38.5	46.2	52.8	55.0	18.8	24.6	27.5	33.3	37.7
LeanSearch v2 (reasoning)	36.7	46.1	50.8	56.4	57.1	27.5	30.4	34.8	43.5	43.5
<i>Premise selection (proof-state query)</i>										
ReProver	3.0	5.5	7.6	8.8	10.3	0.0	1.4	2.9	4.3	4.3
LeanPremise	4.1	4.8	8.0	9.9	11.5	1.4	1.4	4.3	4.3	5.8
LeanStateSearch	6.5	9.3	12.5	16.0	20.0	2.9	2.9	7.2	10.1	13.0

We compare against two complementary baseline families. The first is the reasoning-intensive retrieval line (INF-X-Retriever [Yao et al., 2025], ReasonIR [Shao et al., 2025], and DIVER [Long et al., 2025b]), which we run over our hierarchy-informalized Mathlib corpus (§3.1) so that they read each declaration in the same dependency-aware natural-language form that LeanSearch v2 uses. All published ablations of INF-X and DIVER are reported individually so that the comparison includes each system at its strongest configuration. The second is the Lean-side premise-selection line, which conditions on the formal proof state of the target theorem (extracted through the Lean 4 REPL; see Appendix B.3): ReProver [Yang et al., 2023], LeanPremise [Zhu et al., 2025], and LeanStateSearch [Tao et al., 2025]. Reasoning retrievers and our reasoning-mode pipeline take the informal description + formal statement as input; the premise-selection systems take the formal proof state, matching their published query contract. Per-system input details are in Appendix B.3.

We report two top- k metrics over $k \in \{5, 10, 20, 30, 50\}$. **Recall (group)** is the fraction of ground-truth premise groups hit (where a group counts as hit if any of its interchangeable lemmas appears in the top- k), macro-averaged over queries. **Covered** is the per-query indicator of whether some complete proof routing (the original PR routing or an expert-annotated alternative) has *every* premise group hit. An ablation that uses only the original PR routing as the success criterion is reported in Appendix C.3.

LeanSearch v2 (reasoning) leads every reported cell of Table 2: against the strongest reasoning baseline (the DIVER full pipeline), v2 improves group-recall@10 by +8.1 points (46.1 vs. 38.0) and Covered@10 by +5.8 points (30.4 vs. 24.6). This margin reflects two design choices: reasoning mode plans at the granularity of an overall proof strategy rather than the next tactic step, yielding sub-queries that better match retrieval intent; and the judge-reviser loop re-routes the sketch when the corpus cannot support a planned premise, filtering out routes that are valid in principle but un-coverable in practice. Within the reasoning-retrieval block, the INF-X query rewriter underperforms the bare INF-X retriever (group-recall@10 of 17.4 vs. 28.2): the rewriter is trained to distill verbose, reasoning-heavy queries into short search phrases, but our queries are already a concise formal statement plus informal description, so the distillation has little useful work to do and tends to introduce paraphrasing drift. We also observe that within the reasoning-retrieval block the DIVER full pipeline sits closest to our reasoning-mode result—an outcome consistent with its own multi-stage design (query expansion, retrieval, listwise reranking) investing substantial inference-time LLM capacity, so the comparison runs in a regime where both sides commit comparable inference-time resources. The premise-selection block recovers proportionally fewer premise groups across all k , which we read as a scope-mismatch effect rather than a system-quality gap: the premise-selection systems focus on finding premises that advance a local proof state, and the set-valued target of

Table 3: **Prove task: end-to-end proof success under a fixed simple reflection loop, varying only the retriever.** *#solved* counts problems for which the workflow produced a Lean-verified, sorry-free proof within 8 reflection rounds. FATE-H [Jiang et al., 2025] contains 100 problems; MathlibMPR-Prop is the 50-problem subset of MathlibMPR consisting of theorems whose formal statement admits the uniform tactic-mode `:= by sorry` format (Appendix A.2). Best per column in **bold**.

Retriever	FATE-H ($n=100$)	MathlibMPR-Prop ($n=50$)
no retrieval	4 (4.0%)	2 (4.0%)
LeanFinder	12 (12.0%)	5 (10.0%)
LeanStateSearch	7 (7.0%)	3 (6.0%)
INF-X-Retriever	16 (16.0%)	5 (10.0%)
LeanSearch v2 (standard mode)	14 (14.0%)	5 (10.0%)
LeanSearch v2 (reasoning mode)	20 (20.0%)	7 (14.0%)

global premise retrieval falls outside that intended scope. We include them for completeness as the dominant Lean-side retrieval line.

4.3 Prove: downstream proof-generation evidence

The Prove task tests whether the retrieval-quality ordering observed in §4.2 carries over to end-to-end proof generation. We adopt a fixed proof-generation workflow — the *simple reflection loop* — in which a single prover LLM attempts the goal, and on each compile failure the workflow routes the verifier trace through a retrieve-and-reflect step before the next attempt. Across all configurations the prover and reflection LLM are Claude Sonnet 4.5, running with 8 reflection rounds; the criterion for counting a proof as solved is detailed in Appendix B.4. We evaluate on FATE-H [Jiang et al., 2025] (100 algebra problems sampled across sub-fields) and on *MathlibMPR-Prop* (50 problems), the subset of MathlibMPR consisting of theorems whose formal statement admits the uniform tactic-mode format `:= by sorry` (Appendix A.2), and which spans the same range of mathematical subjects as the parent benchmark.

We compare six configurations: (i) **no retrieval** (prover sees only the goal); (ii) **LeanFinder**; (iii) **LeanSearch v2 standard mode**; (iv) **LeanStateSearch**; (v) **INF-X-Retriever**; and (vi) **LeanSearch v2 reasoning mode**. Each retriever is queried in the input format that matches its published contract: the Lean-side semantic search engines (LeanFinder, LeanSearch v2 standard mode) accept a natural-language reformulation of the current goal; LeanStateSearch accepts the formal proof state taken from the Lean 4 REPL at the first error in the prover’s current attempt; INF-X-Retriever and LeanSearch v2 reasoning mode accept the informal description + formal statement of the theorem. The semantic search engines fire only during reflection, since their queries depend on the prover’s current attempt; the retrievers that condition on the theorem statement fire both at the initial attempt and during reflection. Full input handling is documented in Appendix B.4.

LeanSearch v2 reasoning mode (Table 3, last row) attains the highest success rate on both datasets (20.0% on FATE-H, 14.0% on MathlibMPR-Prop), and the relative ordering among the four retrievers that condition on the theorem statement matches the MathlibMPR group-recall ranking from Table 2. The two Lean-side semantic search engines, queried only at reflection time, sit between the no-retrieval baseline and the retrievers that see the full theorem statement up front. This concludes our evaluation: retrieval quality at the candidate-set level propagates to end-to-end proof success rather than being attenuated by the prover and reflection loop, with the relative downstream ordering matching the upstream retrieval ordering across datasets.

5 Limitations

First, we evaluate exclusively on Lean 4/Mathlib. The methodology is language-agnostic in design, but empirical transfer to other provers (Coq, Isabelle) with different library conventions remains to be demonstrated. Second, our downstream experiment isolates retrieval quality by using a minimal reflection loop workflow designed to prove a single theorem at a time. How the system behaves inside highly capable agent systems [Chen et al., 2025, Achim et al., 2025, Ju et al., 2026] on complex tasks such as repository-level formalization, where retrieval interacts with planning and tactic generation, is unexplored. Third, we focus on retrieval for proof construction. Other retrieval-

intensive Lean tasks, such as locating canonical definitions or navigating the type-class hierarchy during statement construction, may require different query formulations and evaluation criteria and are not studied here.

6 Conclusion

We formulated *global premise retrieval* and presented LeanSearch v2, a staged system whose standard mode achieves state-of-the-art Mathlib search without domain-specific fine-tuning and whose reasoning mode targets set-valued premise retrieval through iterative sketch-retrieve-reflect cycles. Two expert-curated benchmarks, MathlibQR and MathlibMPR, provide the evaluation foundation; to our knowledge, this work is the first to bring reasoning-retrieval techniques to the Lean premise retrieval problem. Reasoning mode leads all baselines on MathlibMPR, and a controlled downstream experiment confirms that this advantage propagates to proof success. We hope the task formulation, benchmarks, and system presented here help establish retrieval as a distinct axis of progress for formal mathematics.

Acknowledgments

This work is supported in part by the National Key R&D Program of China grant 2024YFA1014000, the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (JYB2025XDXM113), and the New Cornerstone Investigator Program.

References

- Abdelrahman Abdallah, Mohamed Darwish Mounis, Mahmoud Abdalla, Mahmoud SalahEldin Kasem, Mostafa Farouk Senussi, Mohamed Mahmoud, Mohammed Ali, Adam Jatowt, and Hyun-Soo Kang. Mm-bright: A multi-task multimodal benchmark for reasoning-intensive retrieval. *arXiv preprint arXiv:2601.09562*, 2026.
- Tudor Achim, Alex Best, Alberto Bietti, Kevin Der, Mathis Fédérico, Sergei Gukov, Daniel Halpern-Leistner, Kirsten Henningsgard, Yury Kudryashov, Alexander Meiburg, et al. Aristotle: Imo-level automated theorem proving. *arXiv preprint arXiv:2510.01346*, 2025.
- Anthropic. System card: Claude Sonnet 4.5. <https://www.anthropic.com/claude-sonnet-4-5-system-card>, 2025.
- Justin Asher. Leanexplore: A search engine for lean 4 declarations. *arXiv preprint arXiv:2506.11085*, 2025.
- Jeremy Avigad and Patrick Massot. Mathematics in lean, 2020.
- Jiangjie Chen, Wenxiang Chen, Jiacheng Du, Jinyi Hu, Zhicheng Jiang, Allan Jie, Xiaoran Jin, Xing Jin, Chenggang Li, Wenlei Shi, et al. Seed-prover 1.5: Mastering undergraduate-level theorem proving via learning from experience. *arXiv preprint arXiv:2512.17260*, 2025.
- Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025.
- Debrup Das, Sam O’Nuallain, and Razieh Rahimi. Rader: Reasoning-aware dense retrieval models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 19981–20008, 2025.
- Guoxiong Gao, Haocheng Ju, Jiedong Jiang, Zihan Qin, and Bin Dong. A semantic search engine for mathlib4. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 8001–8013, 2024a.
- Guoxiong Gao, Yutong Wang, Jiedong Jiang, Qi Gao, Zihan Qin, Tianyi Xu, and Bin Dong. Herald: A natural language annotated lean 4 dataset. *arXiv preprint arXiv:2410.10878*, 2024b.

- Thomas Hubert, Rishi Mehta, Laurent Sartran, Miklós Z Horváth, Goran Žužić, Eric Wieser, Aja Huang, Julian Schrittwieser, Yannick Schroecker, Hussain Masoom, et al. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 651:607–613, 2025. doi: 10.1038/s41586-025-09833-y.
- Jiedong Jiang, Wanyi He, Yuefeng Wang, Guoxiong Gao, Yongle Hu, Jingting Wang, Nailin Guan, Peihao Wu, Chunbo Dai, Liang Xiao, et al. Fate: A formal benchmark series for frontier algebra of multiple difficulty levels. *arXiv preprint arXiv:2511.02872*, 2025.
- Haocheng Ju and Bin Dong. Mirb: Mathematical information retrieval benchmark. *arXiv preprint arXiv:2505.15585*, 2025.
- Haocheng Ju, Guoxiong Gao, Jiedong Jiang, Bin Wu, Zeming Sun, Leheng Chen, Yutong Wang, Yuefeng Wang, Zichen Wang, Wanyi He, et al. Automated conjecture resolution with formal verification. *arXiv preprint arXiv:2604.03789*, 2026.
- Julian Killingback and Hamed Zamani. Benchmarking information retrieval models on complex retrieval tasks. *arXiv preprint arXiv:2509.07253*, 2025.
- Junqi Liu, Zihao Zhou, Zekai Zhu, Marco Dos Santos, Weikun He, Jiawei Liu, Ran Wang, Yunzhou Xie, Junqiao Zhao, Qiufeng Wang, et al. Numina-lean-agent: An open and general agentic reasoning system for formal mathematics. *arXiv preprint arXiv:2601.14027*, 2026.
- Meixiu Long, Duolin Sun, Dan Yang, Yihan Jiao, Lei Liu, Jiahai Wang, BinBin Hu, Yue Shen, Jie Feng, Zhehao Tan, et al. Grouprank: A groupwise paradigm for effective and efficient passage reranking with llms. *arXiv:2511.11653*, 2025a.
- Meixiu Long, Duolin Sun, Dan Yang, Junjie Wang, Yecheng Luo, Yue Shen, Jian Wang, Hualei Zhou, Chunxiao Guo, Peng Wei, et al. Diver: A multi-stage approach for reasoning-intensive information retrieval. *arXiv preprint arXiv:2508.07995*, 2025b.
- Jialin Lu, Kye Emond, Kaiyu Yang, Swarat Chaudhuri, Weiran Sun, and Wuyang Chen. Lean finder: Semantic search for mathlib that understands user intents. *arXiv preprint arXiv:2510.15940*, 2025.
- Mathlib Community. Mathlib contribution guidelines. <https://leanprover-community.github.io/contribute/>, 2024.
- Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. Mteb: Massive text embedding benchmark. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, pages 2014–2037, 2023.
- Job Petrovčič, David Eliecer Narvaez Denis, and Ljupčo Todorovski. Combining textual and structural information for premise selection in lean. *arXiv preprint arXiv:2510.23637*, 2025.
- Auguste Poiroux, Viktor Kuncak, and Antoine Bosselut. Leaninteract: A python interface for lean 4. <https://github.com/augustepoiroux/LeanInteract>, 2025.
- ZZ Ren, Zhihong Shao, Junxiao Song, Huajian Xin, Haocheng Wang, Wanjia Zhao, Liye Zhang, Zhe Fu, Qihao Zhu, Dejian Yang, et al. Deepseek-prover-v2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. *arXiv preprint arXiv:2504.21801*, 2025.
- Rulin Shao, Rui Qiao, Varsha Kishore, Niklas Muennighoff, Xi Victoria Lin, Daniela Rus, Bryan Kian Hsiang Low, Sewon Min, Wen-tau Yih, Pang Wei Koh, et al. Reasonir: Training retrievers for reasoning tasks. *arXiv preprint arXiv:2504.20595*, 2025.
- Ziju Shen, Naohao Huang, Fanyi Yang, Yutong Wang, Guoxiong Gao, Tianyi Xu, Jiedong Jiang, Wanyi He, Pu Yang, Mengzhou Sun, et al. Real-prover: Retrieval augmented lean prover for mathematical reasoning. *arXiv preprint arXiv:2505.20613*, 2025.
- Hongjin Su, Howard Yen, Mengzhou Xia, Weijia Shi, Niklas Muennighoff, Han-yu Wang, Haisu Liu, Quan Shi, Zachary S Siegel, Michael Tang, et al. Bright: A realistic and challenging benchmark for reasoning-intensive retrieval. *arXiv preprint arXiv:2407.12883*, 2024.

- Yicheng Tao, Haotian Liu, Shanwen Wang, and Hongteng Xu. Learning an effective premise retrieval model for efficient mathematical formalization. *arXiv preprint arXiv:2501.13959*, 2025.
- Kimi Team, Yifan Bai, Yiping Bao, Y Charles, Cheng Chen, Guanduo Chen, Haiting Chen, Huarong Chen, Jiahao Chen, Ningxin Chen, et al. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025.
- The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020. doi: 10.1145/3372885.3373824.
- Sean Welleck, Jiacheng Liu, Ronan Le Bras, Hannaneh Hajishirzi, Yejin Choi, and Kyunghyun Cho. Naturalproofs: Mathematical theorem proving in natural language. *arXiv preprint arXiv:2104.01112*, 2021.
- Orion Weller, Kathryn Ricci, Eugene Yang, Andrew Yates, Dawn Lawrie, and Benjamin Van Durme. Rank1: Test-time compute for reranking in information retrieval. *arXiv preprint arXiv:2502.18418*, 2025.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Kaiyu Yang, Aidan Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan J Prenger, and Animashree Anandkumar. Leandojo: Theorem proving with retrieval-augmented language models. *Advances in Neural Information Processing Systems*, 36:21573–21612, 2023.
- Yichen Yao, Jiahe Wan, Yuxin Hong, Mengna Zhang, Junhan Yang, Zhouyu Jiang, Qing Xu, Kuan Lu, Yinghui Xu, Wei Chu, Emma Wang, and Yuan Qi. Inf-x-retriever: A pragmatic framework for reasoning-intensive dense retrieval, 2025. URL <https://github.com/yaoyichen/INF-X-Retriever>.
- Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, et al. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*, 2025.
- Thomas Zhu, Joshua Clune, Jeremy Avigad, Albert Qiaochu Jiang, and Sean Welleck. Premise selection for a lean hammer. *arXiv preprint arXiv:2506.07477*, 2025.

A Benchmark Curation

In this section, we present the curation methodology, key characteristics, and illustrative examples of our benchmark to provide readers with a clearer understanding of both the benchmark and the experimental results. We introduce two benchmarks: **MathlibQR** for the classical formal theorem searching task, and **MathlibMPR** for global premise retrieval. Detailed descriptions are provided in the appendix: MathlibQR is discussed in Appendix A.1, and MathlibMPR in Appendix A.2.

A.1 Searching Mathlib theorems: MathlibQR

MathlibQR is designed for classical theorem retrieval tasks: given a user query, the goal is to select the most relevant formal theorem (document).

A.1.1 Curation and characteristics

The benchmark comprises 200 documents and 946 queries, where multiple queries may correspond to a single document. This design reflects the diversity of human or AI generated search queries. Each document can be associated with up to six distinct query styles, namely Lean-centric, LaTeX expressions, pure natural language, slogan, nickname, and special instances, with at most one query per style. This variety specifically captures the heterogeneity of Lean users with different levels of familiarity with Lean declarations and abstraction, as well as mathematicians of varying maturity, who may express queries using natural language, LaTeX formulas, slogans, or nicknames.

The last category, *special instances*, addresses a common but previously underexamined challenge: users often have a specific application scenario in mind without knowing the exact formulation of the corresponding statement in Mathlib. For instance, a user may wish to apply a theorem about convex sets in $\mathbb{R}^n$ without being aware of its generalization in Mathlib. In this case, the actual condition replacing $\mathbb{R}^n$ involves `[Semiring k] [PartialOrder k] [AddCommMonoid E] [SMul k E]`, which requires expert formalization knowledge to anticipate. A well-designed formal theorem search engine should return relevant results even in such demanding cases. Additional examples of different query styles are provided in Appendix A.1.2.

In terms of coverage, the benchmark spans a wide range of mathematical domains—including algebra, analysis, logic, geometry, probability, number theory, topology, and category theory—and covers all major Mathlib declaration types, such as theorems, definitions, and instances. Each document is further classified into two difficulty levels, easy and hard, based on two dimensions: mathematical obscurity and technical complexity.

To construct such a broadly representative benchmark, we first asked formalization experts to select 8 documents from each of 25 top-level Mathlib folders (out of 31 total, excluding `Control`, `Deprecated`, `Lean`, `Tactic`, `Testing`, and `Util`), ensuring balanced coverage across declaration types and difficulty levels. After finalizing the 200 documents, the experts produced up to six query styles per document, wherever a given style was applicable.

In total, we collected 946 queries (detailed statistics are provided in Table 4), based on Mathlib version v4.29.1, the latest release at the time of construction. As Mathlib is continuously evolving, the corpora of LeanFinder, LeanExplore and this work are built on different versions. Consequently, a subset of 171 documents with 810 queries falls within the common corpus shared by all search tools compared in our experiments (Section 4.1).

A.1.2 Examples

These examples illustrate how a mathematical result can be expressed through diverse query styles, reflecting the varied ways users—from formalization experts to mathematicians—might search for the same theorem. Each document in the benchmark is paired with four to six such styles, capturing the heterogeneity of search intents in practice.

Example 1. `Finset.all_card_le_biUnion_card_iff_existsInjective'`. Hall’s marriage theorem, a classic result in combinatorics, states that a perfect matching exists between n men and n women if and only if every k -subset of men collectively likes at least k women. Combinatorial

Table 4: Benchmark statistics of MathlibQR across the 200 entries.

(a) By query type

	Lean	LaTeX	Natural	Slogan	Nickname	Special Case
Query Number	199	200	199	197	128	23

(b) By difficulty

	Easy	Hard
Document Number	101	99

theorems of this kind frequently arise in application-oriented contexts, motivating us to include a concrete special case as a query.

1. `Finset.all_card_le_biUnion_card_iff_existsInjective'`

Lean	$(\forall s : \text{Finset } \iota, \#s \leq \#(s.\text{biUnion } t)) \iff \exists f : \iota \rightarrow \alpha, \text{Injective } f \wedge \forall x, f x \in t x$
LaTeX	$\forall S, S \leq \bigcup_{i \in S} t(i) \iff \exists f \text{ injective}, f(i) \in t(i)$
Natural	A finite system has a system of distinct representatives iff every subfamily covers enough elements
Slogan	Hall's condition characterizes existence of an SDR
Nickname	Hall's marriage theorem
Special case	n men and n women have a perfect matching iff every k -subset of men collectively likes at least k women

Example 2. `exists_continuous_zero_one_of_isClosed`. Urysohn's lemma states that for any two disjoint closed sets S and T in a normal topological space X , there exists a continuous function $f : X \rightarrow \mathbb{R}$ such that $f|_S = 0$, $f|_T = 1$, and $0 \leq f(x) \leq 1$ for all $x \in X$. This theorem is widely recognized both by its name and its slogan. We also include a special case applying it to the real numbers—a query style for which recovering the general normal-space formulation is nontrivial.

2. `exists_continuous_zero_one_of_isClosed`

Lean	for disjoint closed s t in a <code>NormalSpace</code> , exists continuous f with $f = 0$ on s , $f = 1$ on t
LaTeX	$\exists f \in C(X, [0, 1]), f _S = 0, f _T = 1$ for S, T disjoint closed in normal X
Natural	In a normal space, two disjoint closed sets can be separated by a continuous function valued 0 on one and 1 on the other
Slogan	Disjoint closed sets in a normal space are separated by a continuous function
Nickname	Urysohn's lemma
Special case	In $\mathbb{R}$, $[0, 1]$ and $[2, 3]$ are separated by a continuous ramp

Example 3. `AlexandrovDiscrete`. An Alexandrov-discrete space (or Alexandrov topology) is a topological space in which the intersection of every family of open sets is open—a condition strictly weaker than discreteness. Since this name is less familiar than the property itself, the definition is more naturally searched for via descriptive styles (Lean, LaTeX, natural language) than by nickname.

Example 4. `Condensed.instAB4CondensedMod`. Some statements are stored as instances rather than theorems. The slogan "CondensedMod satisfies AB4" captures one such example. We also include the Lean query instance : `AB4 (CondensedMod R)`, which is the most direct way for formalization experts to search for this result.

3. AlexandrovDiscrete

Lean	a <code>TopologicalSpace</code> where $\bigcap_0 S$ is open for any family S of opens
LaTeX	$\bigcap_{i \in I} U_i$ open for arbitrary index set I
Natural	A topological space where intersections of arbitrary open families remain open
Slogan	Arbitrary intersections of opens stay open
Nickname	Alexandrov-discrete space
Special case	—

4. Condensed.instAB4CondensedMod

Lean	<code>instance : AB4 (CondensedMod R)</code>
LaTeX	<code>CondensedMod R</code> satisfies Grothendieck axiom AB4
Natural	Condensed modules satisfy Grothendieck AB4 (infinite direct sums are exact)
Slogan	<code>CondensedMod</code> satisfies AB4
Nickname	—
Special case	—

A.2 Global premise retrieval: MathlibMPR

MathlibMPR is a benchmark for global premise retrieval: given a mathematical problem, the task is to retrieve the theorems representing the key mathematical steps that, when combined, yield a solution. The name MathlibMPR derives from its construction method—the benchmark is built from merged pull requests in Mathlib.

A.2.1 Curation and characteristics

MathlibMPR contains 69 mathematical problems extracted from merged Mathlib pull requests. Each problem is equipped with a natural language statement, a formal statement, and a list of premise groups. Each premise group consists of several lemmas that are mathematically equivalent but differ in their formalizations, and is tagged as either *original* or *alternative*. The original tag indicates lemmas that appear as vital steps in the proof merged into Mathlib, while the alternative tag indicates lemmas that play an essential role in a different, equally viable proof.

During curation, formalization experts selected merged PRs that postdate the corpora of all search engines compared in our experiments (Section 4.2). For each selected PR, experts identified a main theorem, summarized the key mathematical steps, and annotated the corresponding Mathlib declarations. When possible, they also constructed alternative proofs viable within Mathlib and annotated the lemmas constituting the key steps of those proofs.

All selected PRs were merged more than six months after the knowledge cutoff of the prover model used in our experiments (Claude Sonnet 4.5, Section 4.3), ensuring no data contamination.

To ensure compatibility with the prover pipeline, we further filtered the problems down to 50 formal statements with a uniform format (each ending in `:= by sorry`), allowing all premise selection tools to function correctly.

A.2.2 Examples

In this section, we highlight several key properties of global premise retrieval: navigating typeclass inheritance chains, proposing intermediate reduction steps, accommodating multiple viable proof strategies, and using abstract machinery. Some of these properties are shared with general reasoning retrieval tasks, while others are distinctive to formal mathematics; we will provide a detailed comparison later in Appendix A.2.3.

Example 1 (Intermediate reduction steps). PR 30112 proves Kallenberg’s “isolation of randomness” lemma: given a standard Borel space Y and a Markov kernel $\kappa : X \rightsquigarrow Y$, there exists a jointly measurable map $f : X \rightarrow I \rightarrow Y$ such that for every $a : X$, the pushforward of the uniform measure on the unit interval I via $f(a, \cdot)$ equals $\kappa(a, \cdot)$.

The proof requires first reducing to the case $Y = I$, using the fact that every Markov kernel into a standard Borel space can be represented as the pushforward of the uniform distribution on $[0, 1]$ under a family of jointly measurable functions. Without this reduction, the technical lemmas for constructing the inverse CDF are inapplicable. This reduction step is highly nontrivial: it does not emerge naturally from the original statement and demands substantial mathematical experience and, often, multiple reasoning attempts to discover.

Example 2 (Multiple proof strategies). PR 36490 proves the following statement in category theory: given an adjunction $F \dashv G : \mathcal{C} \rightleftarrows \mathcal{D}$ with Grothendieck topologies J on $\mathcal{C}$ and K on $\mathcal{D}$, the left adjoint F is cocontinuous from J to K (every K -cover of FU pulls back to a J -cover of U) if and only if G preserves covers from K to J (every K -cover of X is sent to a J -cover of GX).

The proof merged in the PR proceeds by elementary manipulations of sieves using the unit and counit of the adjunction. A conceptually cleaner alternative is available: F is cocontinuous if and only if its pullback G^* preserves sheaves, which holds if and only if G preserves covers. This route is more elegant but requires substantial sheafification machinery beyond what is currently available in Mathlib, making it impractical to execute within a reasonable proof length. Consequently, the elementary approach was adopted.

Example 3 (Abstract machinery). PR 31754 proves that every non-empty light profinite space is an injective object in the category of profinite spaces: for S light profinite and non-empty, any commutative square

$$\begin{array}{ccc} X & \hookrightarrow & Y \\ g \downarrow & \swarrow & \\ S & & \end{array}$$

with $X \hookrightarrow Y$ a monomorphism in the category of profinite spaces admits a lift $Y \rightarrow S$ extending g .

The proof proceeds in two stages. First, one establishes the result for finite spaces S and maps between finite spaces. Then, writing any light profinite space S as a cofiltered sequential limit of finite spaces, one applies the abstract machinery of “passing to limits” to transfer the result to the general case. Both preliminary steps are specifically crafted to feed into this abstract argument: without recognizing that the problem calls for passage to a cofiltered limit, the individual lemmas serve no purpose. Solving this problem thus demands the ability to abstract from the concrete statement to the framework of cofiltered limits—a step that depends crucially on the capacity to recognize the right abstract structure underlying a concrete problem.

A.2.3 Comparison with reasoning retrieval tasks

The previous section highlighted several challenges unique to global premise retrieval: proposing intermediate reduction steps, choosing among multiple proof strategies, and using abstract machinery. In this section, we compare these characteristics with those of BRIGHT [Su et al., 2024], a traditional reasoning retrieval benchmark. The following two examples are drawn from its appendix.

Example 1 (Table 27 in Su et al. [2024]). This example is drawn from a LeetCode problem:

Query:

Given ‘n’ non-negative integers representing an elevation map where the width of each bar is ‘1’, compute how much water it can trap after raining.

Example 1:

Input: height = [0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]

Output: 6

Explanation: The above elevation map (black section) is represented by array [0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]. In this case, 6 units of rain water (blue section) are being trapped.

Example 2:

Input: height = [4, 2, 0, 3, 2, 5]

Output: 9

Constraints:

- `n == height.length`
- `1 <= n <= 2 * 104`
- `0 <= height[i] <= 105`

Chain-of-thought reasoning to find documents:

This problem can be solved using a two-pointer approach, and uses ideas from dynamic programming to keep track of the maximum height to the left and right of each bar. We can find other example code that also use these techniques.

Analysis of Example 1. This retrieval problem also requires reasoning before retrieval: one must first recall possible solution strategies (e.g., two-pointer techniques, dynamic programming) before searching for relevant documents. However, it lacks the distinctive challenges that appear in global premise retrieval—namely, the need to abstract from a concrete statement to a general framework, or to choose among multiple viable proof strategies based on what machinery is available. The reasoning here relies more on association and recognition than on structural abstraction, making it less demanding in terms of reasoning depth than formal theorem retrieval.

Example 2 (Table 29 in Su et al. [2024]). This example is drawn from 2015 AMC 10B Problem 15:

Query:

The town of Hamlet has 3 people for each horse, 4 sheep for each cow, and 3 ducks for each person. Which of the following could not possibly be the total number of people, horses, sheep, cows, and ducks in Hamlet? (A) 41 (B) 47 (C) 59 (D) 61 (E) 66

Chain-of-thought reasoning to find documents:

We can use the Chicken McNugget Theorem to solve this problem. We can find other solutions that also apply this theorem.

Analysis of Example 2. This retrieval problem hinges on a single, well-known theorem: once the solver recognizes the problem as an application of the Chicken McNugget Theorem, the retrieval target is essentially fixed. There is no need to find the reduction steps, evaluate multiple viable proof strategies, or lift a concrete statement to an abstract framework. The reasoning required in this task is considerably less demanding than research-level formal premise retrieval in Mathlib.

B Experiment Setup

B.1 Reasoning-mode aggregation across sub-queries

We rank the final outputs of reasoning mode as follows. A reasoning-mode sketch decomposes the target theorem into multiple sub-queries; each sub-query goes through standard mode and is post-processed by the document filter, yielding a per-sub-query ranked list of accepted Mathlib declarations. The same declaration may appear in several sub-queries’ lists, and across sub-queries the raw retrieval scores are not directly comparable, since each sub-query is its own retrieval distribution under a different instruction. We therefore discard the raw scores and aggregate by rank position only.

Concretely, let $L_1, \dots, L_m$ be the per-sub-query filtered lists for an accepted sketch (each L_j is a length- $|L_j|$ list of declaration ids). For each declaration d that appears at 0-indexed rank $i_j(d)$ in L_j we assign a positional contribution $1/\log_2(i_j(d) + 2)$ from list j , and define d ’s aggregate score as

$$s(d) = \sum_{j: d \in L_j} \frac{1}{\log_2(i_j(d) + 2)}.$$

The j -th list contributes 1.00 for its rank-1 entry, 0.63 for rank 2, 0.50 for rank 3, and so on, with the same declaration’s contributions summed across the lists in which it appears. Reasoning mode’s

final ranking is the descending sort of $\{s(d)\}$ over the union of accepted declarations, truncated to top- k .

We adopted this rank-only rule after an initial design that aggregated by taking each declaration’s maximum raw cosine or rerank score across sub-queries. The score-based design was empirically poor at low k : the embedder’s cosine similarities and the reranker’s $P(\text{yes})$ outputs are conditioned on per-sub-query instructions and queries, so scores on different sub-queries occupy incomparable ranges. Replacing raw scores with a position-only discount removes the cross-sub-query calibration problem entirely and gives consistent improvements at small k without sacrificing performance at $k \geq 100$.

B.2 LLM-as-judge protocol for the Search task

The LLM-as-judge column in Table 1 follows the protocol introduced for Mathlib search by Asher [2025], with two modifications below to accommodate a four-system comparison and to mitigate position bias. We briefly explain how the protocol works and what self-checks we ran.

Each retrieved declaration is presented to the judge as a six-field block: fully-qualified Mathlib name, declaration kind, human-readable informal name, formal signature (truncated to 800 characters), value or proof body (truncated to 400 characters), and informalized natural-language statement (truncated to 600 characters). All four systems’ results are hydrated from a single shared metadata source used by standard mode, so the form of the judge’s reading material is identical across systems. For each of the 810 active queries we sample three distinct permutations (without replacement, out of 24 possible) of the four systems via a deterministic per-query seed, and in every prompt the systems are presented anonymously as *System A / B / C / D* so that the judge cannot exploit system identity; the per-query permutations across rounds expose each system to multiple label positions. The judge is Claude Sonnet 4.5, called at `temperature= 0`, `max_tokens= 600`, and prompted to produce a strict total ordering 1–4 over the four top-5 result sets, output as a JSON object whose `ranking` field is a permutation of `["A", "B", "C", "D"]`. Ties are forbidden by prompt; the parser observed no malformed responses across 2430 judgments per system after stripping JSON code fences. For each (slice, system) cell we report mean rank (lower is better; range 1–4) over $n_{\text{slice}} \times 3$ judgments, which on the overall slice is $810 \times 3 = 2430$ judgments per system.

Two self-checks support reading the resulting numbers as stable. The first is cross-round agreement: per-round mean ranks vary by at most a few hundredths of a rank across the three rounds, and the mean pairwise Spearman correlation between the round- i and round- j rank vectors is consistently high. The second is a position-bias check: the judge does exhibit a primacy bias — every system receives a better mean rank when assigned to label position *A* than to *D*, with a swing of approximately 0.42–0.58 ranks (Table 5) — but the permutation procedure assigns each system to each of the four label positions approximately equally often (574–640 assignments per cell on 2430 judgments; uniform expectation is 607.5), so the bias averages out at the system level and the reported overall mean ranks are debiased.

Table 5: Mean rank conditioned on the system’s randomized label position. Values are means over the queries assigned to that position by the per-query seed.

System	Position A	Position B	Position C	Position D
LeanSearch v2 (rerank)	1.290	1.642	1.718	1.865
LeanSearch v2 (retriever only)	1.854	2.156	2.327	2.370
LeanFinder	2.522	2.877	3.091	2.985
LeanExplore	3.011	3.379	3.479	3.429

B.3 Global premise retrieval: per-system input and output handling

The eight reasoning-retrieval rows and three premise-selection rows in Table 2 differ in two procedural respects: what they receive as the per-query input, and how their output is post-processed before scoring. Within each family all systems produce a top-100 list of Mathlib declaration ids, and Mathlib declaration names are unique once normalised, so output handling is not a per-system distinction but a uniform doc-id match. Input, however, splits into three families — our reasoning-mode pipeline, the reasoning retrievers, and the premise-selection systems — and we describe each family in turn. Table 6 summarises the resulting per-row pipelines.

LeanSearch v2 reasoning mode. We treat reasoning mode (§3.2) as a black box here: input is the raw query field — the concatenation of the informal description and the formal Lean statement — and output is a ranked list of Mathlib declaration ids of variable length. Mean output length on **MathlibMPR** is 30.6 docs per query (P25 = 21, median = 26, P75 = 38, max = 90); we score the system at every $k \leq 100$ but its $k \geq 50$ cells are nearly identical to its $k = 30$ cells because the system is output-constrained at large k and does not pad to a fixed length.

Reasoning retrievers. The reasoning-retrieval rows take the same raw query as input and feed it directly to the corresponding encoder, except for two systems that wrap the query before retrieval. INF-X-Retriever in its rewriter regime runs `infly/inf-query-aligner` to produce a single short search query before passing it to the `infly/inf-retriever-v1-pro` encoder. DIVER in its `qexpand` regime runs `DeepSeek-R1-Distill-Qwen-14B` as a query-expansion reasoner whose output is concatenated with the raw query and fed to the `Diver-Retriever-4B` encoder. The other INF-X, ReasonIR, and DIVER configurations feed the raw query directly. Output is the encoder’s (or rerank-augmented encoder’s) top-100 over the same Mathlib corpus we use for standard mode.

Premise selection. ReProver, the `lean-premises` encoder, and `LeanStateSearch` all consume a *proof state* (goal + hypothesis context) rather than an informal description, matching their published query contracts. We extract the proof state by appending `:= sorry` to each query’s formal statement and re-running the resulting declaration through the Lean 4 REPL via `LeanInteract` [Poiroux et al., 2025]: the REPL emits a `Sorry` record whose `goal` field is the target proof state. Coverage on **MathlibMPR** is 69/69 queries successfully extracted. Each system retrieves the top-100 from its own preferred premise corpus.

Table 6: **Per-system pipelines on MathlibMPR.** *Input:* I/F = informal + formal statement; PS = extracted proof state. *Corpus:* Mathlib = our informalized Mathlib snapshot; LD = LeanDojo Benchmark premises; LP = the premise corpus released with `LeanPremise` [Zhu et al., 2025]; LSS = `premise-search.com` server-side corpus.

System (regime)	Input	Pipeline	Corpus
INF-X retriever-only	I/F	retrieve top-100	Mathlib
INF-X + rewriter	I/F	query-aligner → retrieve top-100	Mathlib
ReasonIR	I/F	retrieve top-100	Mathlib
DIVER retriever	I/F	retrieve top-100	Mathlib
DIVER + <code>qexpand</code>	I/F	R1-Distill-14B expand → retrieve top-100	Mathlib
DIVER + rerank	I/F	retrieve top-100 → GroupRank-32B listwise rerank	Mathlib
DIVER full pipeline	I/F	<code>qexpand</code> → retrieve → rerank, score-blended	Mathlib
LeanSearch v2 reasoning	I/F	sketch (Sonnet) → <code>leansearch</code> retriever → filter (Kimi K2 Instruct) → judge (Sonnet) → rank-discount aggregate	Mathlib
ReProver	PS	byt5-small encoder → retrieve top-100	LD
LeanPremise	PS	all-distilroberta-v1 ft encoder → retrieve top-100	LP
LeanStateSearch	PS	HTTP API → top-100	LSS

B.4 Prove: per-system input handling within the simple reflection loop

Every row of Table 3 runs the same simple reflection loop: a prover LLM emits a candidate Lean proof, the Lean REPL verifies it, and on a compile failure a reflection step regenerates the proof with the REPL feedback (and, optionally, retrieval hints) appended. Per-row differences live in two slots: what the retriever sees as its query, and at which step of the loop the retriever fires. We describe each row below, then summarise the integration in Table 7. All rows share the same loop hyperparameters — 8 reflection rounds, `prover_max_retries=3`, `verifier_max_retries=3`, `verifier_wait=30` seconds — and the prover LLM is Claude Sonnet 4.5 in every row of Table 3; per-row Kimi K2 Instruct-as-prover ablations are reported in Appendix D.2. A proof counts as solved iff it compiles and contains no live `sorry` (block- and line-comments are stripped before the check).

No retrieval. The prover sees only the informal description and the formal statement on round one, and on each reflection round receives the previous broken proof and the REPL error message.

There is no retriever in this configuration; the row anchors the contribution that retrieval makes when added to the same loop.

LeanSearch v2 (standard mode). On each reflection round, an LLM rewrites the prover’s current attempt into a natural-language query, queries standard mode, and concatenates the top hits into the reflection prompt. The query rewrite is necessary because standard mode is designed to consume natural-language search phrases, whereas the prover’s intermediate state is a partial Lean proof together with compilation errors.

LeanFinder. Mechanically identical to the LeanSearch v2 standard-mode row except that the natural-language query is sent to the LeanFinder Hugging Face Space via `gradio_client`’s `/retrieve` endpoint, and the HTML-table response is parsed into the canonical hit schema before being inserted into the reflection prompt.

LeanStateSearch. On every round (including round one) the loop extracts the formal proof state through the Lean REPL: it appends `:= sorry` to the current attempt, runs it, and reads the goal/hypothesis context from the resulting Sorry record. That state string is sent to the LeanStateSearch HTTP API, and the returned premise list is inserted into the next prover prompt. We do not run a query-rewrite step here because LeanStateSearch is designed and trained to consume formal proof states directly.

INF-X-Retriever. On every round (including round one) the loop sends a raw query string to a local FastAPI service that wraps the INF-X-Retriever encoder. On round one the query is the concatenation of the informal description and the formal statement, without the rewriter provided by INF-X-Retriever (see §4.2 for the rewriter’s effect on retrieval quality); on reflection rounds the query is the concatenation of the current Lean code and the REPL error message. The service returns doc ids only, which the loop hydrates against our metadata pickle to fill in the formal signature and informalized statement that the prover prompt expects.

LeanSearch v2 (reasoning mode). On round one the loop runs reasoning mode (§3.2) on the informal description + formal statement, producing a sketch and an aggregated premise set; the sketch text and the aggregated premise list are both inserted into the prover prompt. On reflection rounds the loop falls back to the standard-mode reflection path: an LLM rewrites the prover’s current attempt into a natural-language query, queries standard mode, and feeds the hits back. We do not re-invoke reasoning mode at every reflection round because the cost would be prohibitive; the reasoning-mode output is computed once and treated as a fixed sketch that the prover refines against.

Table 7: **Retriever integration across Prove-task rows.** *Round 1:* input to retriever on the first prover call. *Reflection:* input on subsequent rounds after a compile failure. “–” indicates no retrieval call at that step. *informal/formal* = the theorem’s informal description + formal statement; *NL queries* = an LLM-produced natural-language query distilled from the prover’s current attempt; *proof state* = the formal proof state extracted from the Lean 4 REPL; *code+err* = the prover’s current attempt concatenated with the REPL error message.

Row	Round-1 input	Reflection input	Notes
no retrieval	–	–	
LeanSearch v2 (standard mode)	–	NL queries	retriever fires only on reflection
LeanFinder	–	NL queries	gradio_client to HF Space
LeanStateSearch	proof state	proof state	proof state extracted at every round
INF-X-Retriever	informal + formal	code + err	raw query, BRIGHT instruction
LeanSearch v2 (reasoning)	informal + formal	NL queries	reasoning sketch frozen after round 1

C More Experiment Results

C.1 Reasoning-mode loop iterations on MathlibMPR

We instrument reasoning mode on the **MathlibMPR** benchmark to record, per query, which initial-or-revised sketch the judge accepted (if any), how many revisions were required, and whether the parallel branches agreed. Each query is run with `budget = 2` branches in parallel and a per-branch

revision cap of 3, so each branch issues an initial sketch and up to three revised sketches before timing out. The two statistics we report are the revision count of the accepted sketch and the joint outcome of the two branches.

The judge accepts at least one branch’s sketch for 46 of the 69 queries (66.7%); the remaining 23 queries (33.3%) reach the revision cap on every branch, in which case the per-sub-query filtered lists from both branches are pooled by the rank-discount rule of Appendix B.1. Among the 46 accepted queries, the winning branch’s revision count is distributed as in Table 8: nearly two-thirds of accepted queries are accepted on the initial sketch with no revision, and no accepted query requires the third revision.

Table 8: **Revision count of the accepted sketch.** *Initial* = sketch generator only; *revised once / twice* = sketch reviser invoked one or two times before acceptance. Among the 23 unaccepted queries every branch exhausts the cap (revised three times).

Revision count of accepted sketch	# queries	% of 69
0 (initial sketch accepted)	29	42.0%
1 (one revision before acceptance)	11	15.9%
2 (two revisions before acceptance)	6	8.7%
<i>Unaccepted (no branch accepted within 3 revisions)</i>	23	33.3%

At the branch level, individual branches independently terminate as *good* (judge-accepted), *early_stop* (the runtime detected that a sibling’s accepted output already satisfied the budget and halted further work), or *fail* (revision cap exhausted without acceptance). The joint outcomes across the two branches are listed in Table 9, and among the 46 accepted queries branch 0 produced the winning sketch on 31 queries while branch 1 produced it on 15 — a slight imbalance consistent with branch 0 being initialised first and so more often reaching the judge before *early_stop* fires on its sibling.

Table 9: **Joint branch-status distribution.** Branch order is the launch order; *early_stop* indicates the runtime cancelled the branch once a sibling produced an accepted sketch.

(Branch 0, Branch 1) outcome	# queries
(good, good)	24
(early_stop, good)	14
(good, early_stop)	6
(good, fail)	1
(fail, good)	1
(fail, fail)	23

Read together, the two distributions support the loop’s design choices. The 66.7% acceptance rate is comparable to the strongest reasoning-baseline Covered rate at $k = 100$ on MathlibMPR, indicating that the judge’s binary feasibility decision is a reasonable proxy for whether the underlying retrieval substrate has actually surfaced the right premises. The fact that no accepted query requires the maximum number of revisions also suggests the revision cap is appropriately set: each additional revision past the initial sketch yields fewer acceptances ($29 \rightarrow 11 \rightarrow 6$), so the marginal yield of a fourth revision would be small. Finally, the two branches rarely diverge in their final verdict — $24 + 23 = 47$ queries see both branches finish in the same state, ignoring *early_stop* — supporting the design choice of two parallel branches as a low-cost variance-reduction mechanism rather than a substantively different exploration strategy.

C.2 Search task: per-slice breakdown

We report nDCG@10 and Recall@10 on three slicings of the fair (810-row) subset of **MathlibQR**: by query difficulty (Easy / Hard), by ground-truth declaration kind (reporting the kinds for which our encoder uses a kind-specific prompt template, alongside the overall row), and by query style (six styles). Computing the slices on the fair subset keeps the per-slice basis identical to the main-report basis in Table 1. Throughout the following tables we abbreviate column headers as **LE** = LeanExplore, **LF** = LeanFinder, **LSv2(r)** = LeanSearch v2 retriever-only, and **LSv2(rr)** = LeanSearch v2 with reranking.

Table 10: **MathlibQR by difficulty**. Half the queries are Easy, half are Hard.

Difficulty	nDCG@10				Recall@10			
	LE	LF	LSv2(r)	LSv2(rr)	LE	LF	LSv2(r)	LSv2(rr)
Easy	0.410	0.568	0.533	0.666	0.599	0.731	0.722	0.829
Hard	0.373	0.491	0.449	0.571	0.534	0.658	0.580	0.722

Table 11: **MathlibQR by ground-truth declaration kind on the fair subset**. We report the score on the kinds for which our encoder uses a kind-specific prompt template (theorem, def, instance) alongside the overall row, so that the per-kind cells are independent comparisons rather than an exhaustive partition. Best per row in **bold**.

Kind	nDCG@10				Recall@10			
	LE	LF	LSv2(r)	LSv2(rr)	LE	LF	LSv2(r)	LSv2(rr)
overall	0.393	0.533	0.494	0.623	0.569	0.698	0.657	0.780
theorem	0.351	0.617	0.619	0.765	0.546	0.790	0.849	0.914
def	0.441	0.637	0.657	0.764	0.571	0.790	0.829	0.943
instance	0.390	0.661	0.839	0.846	0.619	0.866	0.907	0.938

LeanSearch v2 with reranking leads on every reported slice $\times$ metric cell except a near-tie on q1a Lean queries, where LeanFinder edges out v2 by 0.006 in Recall@10. The lead widens on the harder slices: on *Hard* queries v2 with reranking is +6.4 Recall@10 ahead of the next-best system, and on *q4 special case* — the most adversarial slice for surface matching — the lead is +26.1 Recall@10 over the next-best system in the table. The kind-sliced rows in Table 11 are reported as independent comparisons rather than a partition of the benchmark, since not every declaration kind admits a clean per-kind prompt template; an ablation that isolates the kind-aware instruction’s contribution on the same three kinds is in Appendix D.1.

C.3 Global premise retrieval: alternative success criteria

The headline numbers in Table 2 count a query as solved when *some* expert-annotated proof routing — either the original PR proof or one of the curator’s equivalent alternative routings — has every premise group hit by the top- k . To verify that the conclusions of §4.2 are not driven by the allowance for alternative routings, we report a stricter Covered variant that uses only the original PR routing, alongside the main-text variant on the same set of k values. Table 13 reports both columns over $k \in \{5, 10, 20, 30, 50\}$ for every system.

The relative ordering of systems is preserved between the two variants: LeanSearch v2 reasoning leads in every column, the DIVER full pipeline remains the strongest non-v2 reasoning baseline, and the gap between the strongest premise-selection system at any k and v2 reasoning at $k = 5$ is preserved under both criteria. The absolute gap between Covered (main $\vee$ alt-all) and Covered (main only) is on the order of 1–3 percentage points for the strong reasoning systems and is essentially zero for the premise-selection systems, so the conclusions of §4.2 do not depend on alternative-routing credit.

C.4 MathlibQR across benchmark perspectives

The main report uses the **fair** perspective on **MathlibQR**: every system can in principle return the ground-truth declaration because the query rows are restricted to the intersection of the three Mathlib snapshots (LeanFinder’s, ours, and LeanExplore’s). This is the cleanest comparison, but it discards 136 rows whose ground-truth lives outside one or more competitor snapshots. To check that the conclusions of §4.1 survive on looser comparators, we re-score the same retrievers under two additional perspectives. The *at-least-one* perspective (937 rows) keeps every query whose ground-truth is in at least one of the three snapshots, and the *full* perspective (946 rows) is the unfiltered benchmark, which includes nine rows whose ground-truth is in none of the three snapshots and so cannot be retrieved by any of the four systems.

LeanSearch v2 with reranking ranks first on every (perspective $\times$ metric) cell. The lead over the second-best system is largest on the fair perspective (+0.087 nDCG@5 over LeanFinder, +0.082 Recall@10); on the looser perspectives the second-best slot shifts from LeanFinder to LeanExplore

Table 12: **MathlibQR by query style**. *q1a* Lean-flavored phrasing; *q1b* LaTeX statement; *q1c* plain English statement; *q2* short conceptual slogan; *q3* informal nickname; *q4* special-case instance.

Query style	nDCG@10				Recall@10			
	LE	LF	LSv2(r)	LSv2(rr)	LE	LF	LSv2(r)	LSv2(rr)
q1a Lean	0.340	0.607	0.504	0.622	0.524	0.735	0.624	0.729
q1b LaTeX	0.383	0.551	0.508	0.634	0.561	0.696	0.655	0.807
q1c plain text	0.449	0.528	0.531	0.667	0.647	0.724	0.694	0.824
q2 slogan	0.367	0.441	0.477	0.573	0.518	0.655	0.655	0.720
q3 nickname	0.510	0.607	0.465	0.641	0.685	0.741	0.685	0.843
q4 special case	0.079	0.218	0.317	0.500	0.217	0.348	0.522	0.783

Table 13: **Covered variants on MathlibMPR** ($n=69$, percentages). *Covered (main only)* requires the original PR routing’s groups to all be hit; *Covered (main $\vee$ alt-all)* additionally credits a query when an entire alternative routing’s groups are hit, and is the variant reported in Table 2. Best per column in **bold**.

System	Covered (main only)					Covered (main $\vee$ alt-all)				
	5	10	20	30	50	5	10	20	30	50
<i>Reasoning retrieval (informal + formal query)</i>										
INF-X-Retriever	14.5	17.4	18.8	21.7	21.7	15.9	18.8	21.7	24.6	24.6
+ query rewriter	10.1	10.1	14.5	15.9	23.2	10.1	10.1	17.4	17.4	24.6
ReasonIR	15.9	17.4	18.8	18.8	26.1	17.4	18.8	20.3	21.7	30.4
DIVER (full pipeline)	17.4	21.7	24.6	29.0	31.9	18.8	24.6	29.0	33.3	37.7
retriever only	14.5	14.5	23.2	27.5	27.5	15.9	15.9	26.1	31.9	33.3
+ query expansion only	14.5	15.9	23.2	23.2	26.1	15.9	17.4	26.1	27.5	31.9
+ GroupRank rerank only	17.4	21.7	24.6	29.0	31.9	18.8	24.6	27.5	33.3	37.7
LeanSearch v2 (reasoning)	26.1	29.0	33.3	40.6	40.6	27.5	30.4	34.8	43.5	43.5
<i>Premise selection (proof-state query)</i>										
ReProver	0.0	1.4	2.9	2.9	2.9	0.0	1.4	2.9	4.3	4.3
LeanPremise	1.4	1.4	4.3	4.3	5.8	1.4	1.4	4.3	4.3	5.8
LeanStateSearch	2.9	2.9	7.2	8.7	11.6	2.9	2.9	7.2	10.1	13.0

in some metrics, but our lead remains +0.13 nDCG@5 on at-least-one and +0.13 on full. The absolute scores fall 4–6 points across columns when moving from fair to full, reflecting the extra 136 rows that some systems cannot reach by construction; the relative ordering is unaffected.

D Ablation Study

D.1 Search-task ablations

Standard mode is built up in two design steps: starting from a plain dense retriever, we first add a reranker, and then add a kind-aware instruction to both the corpus encoding and the reranker prompt. Both steps are intended to help, but the contributions are not symmetric and depend on the kind of declaration being retrieved. The ablation in Table 15 isolates each step on the fair subset of **MathlibQR**, breaking the score down on the three declaration kinds for which our encoder uses a kind-specific prompt template (theorem, def, instance) and reporting Overall as the union.

The two design steps stack additively on every slice where headroom exists. On the Overall slice, nDCG@5 climbs from 0.472 at the embedding-only baseline to 0.601 at the production configuration, a total uplift of +0.129 points; the rerank step contributes about three quarters of that gain (+0.101) and the kind step the remaining quarter (+0.029). The breakdown shifts noticeably by kind. On theorem queries the rerank step alone delivers nearly the entire uplift, taking nDCG@5 from 0.593 to 0.746, while the kind step adds ≤ 0.002 on top, indistinguishable from noise on the slice’s 291 queries; this is consistent with the encoder’s default prompt template already being theorem-shaped, so explicitly tagging items as theorems contributes little. On def queries both steps contribute substantively (nDCG@5 0.633 $\rightarrow$ 0.702 $\rightarrow$ 0.738, with the rerank step supplying about two thirds and the kind step the remaining third), making this the clearest example of the two interventions stacking. On instance queries the embedding-only baseline is already at 0.833 nDCG@5

Table 14: **LeanSearch v2 vs. baselines across MathlibQR perspectives.** Each cell is nDCG@ k or Recall@ k on the listed perspective. Best per (perspective $\times$ column) cell in **bold**.

Perspective (n)	System	nDCG@ k			Recall@ k		
		1	5	10	10	50	100
fair (810)	LeanExplore	0.246	0.358	0.393	0.569	0.743	0.789
	LeanFinder	0.370	0.514	0.533	0.698	0.824	0.875
	LeanSearch v2 (rerank)	0.470	0.601	0.623	0.780	0.847	0.858
at-least-one (937)	LeanExplore	0.238	0.351	0.386	0.564	0.735	0.777
	LeanFinder	0.320	0.445	0.461	0.603	0.712	0.757
	LeanSearch v2 (rerank)	0.446	0.576	0.595	0.747	0.810	0.820
full (946)	LeanExplore	0.236	0.347	0.383	0.558	0.728	0.770
	LeanFinder	0.317	0.440	0.457	0.597	0.705	0.750
	LeanSearch v2 (rerank)	0.442	0.570	0.590	0.740	0.802	0.812

Table 15: **Progressive ablation of standard mode on the fair subset of MathlibQR (810 rows).** *no rerank*: Qwen3-Embedding-8B alone. *rerank, no kind*: + Qwen3-Reranker-8B with a kind-blind corpus encoding (every record encoded as theorem) and a kind-agnostic instruction. *rerank, +kind* (production): kind-aware corpus encoding and instruction. Best per (slice $\times$ column) cell in **bold**.

Slice (n)	Pipeline	R@1	R@5	R@10	nDCG@5	nDCG@10
Overall (810)	no rerank	0.340	0.586	0.657	0.472	0.494
	rerank, no kind	0.441	0.694	0.774	0.572	0.598
	rerank, +kind	0.470	0.714	0.780	0.601	0.623
theorem (291)	no rerank	0.381	0.766	0.849	0.593	0.619
	rerank, no kind	0.612	0.856	0.914	0.746	0.765
	rerank, +kind	0.612	0.859	0.914	0.748	0.765
def (105)	no rerank	0.495	0.752	0.829	0.633	0.657
	rerank, no kind	0.505	0.867	0.924	0.702	0.720
	rerank, +kind	0.581	0.867	0.943	0.738	0.764
instance (97)	no rerank	0.773	0.887	0.907	0.833	0.839
	rerank, no kind	0.742	0.887	0.907	0.819	0.825
	rerank, +kind	0.773	0.897	0.938	0.833	0.846

with little headroom; the rerank-no-kind configuration dips by 0.014 before the kind step pulls it back to parity at 0.833, so on this slice the kind step’s role is to prevent a small regression rather than to drive an improvement. Across all four slices, the production rerank-with-kind configuration is at least as good as either of the simpler variants on every metric, and never worse, so the two-step build-up is justified by the data on every reported cell.

D.2 Simple reflection loop with a weaker prover backbone

Table 3 runs the simple reflection loop with Claude Sonnet 4.5 as the prover. We ablate that choice by re-running the same three retrieval modes — no retrieval (L0), LeanSearch v2 standard mode (L1), LeanSearch v2 reasoning mode (the L2 *Sketch* variant) — with Kimi K2 Instruct in the prover role on FATE-H ($n = 100$). All other roles in the loop (the query rewriter on the standard-mode reflection path, the sketch generator / filter / judge inside reasoning mode, and the REPL verifier) are kept identical to the corresponding Sonnet rows of Table 3.

Table 16: **Prover-backbone ablation on FATE-H.** Each cell is the count of solved problems out of 100 (percentages in parentheses). The Sonnet column reproduces the L0 / L1 / reasoning-mode rows of Table 3; the Kimi column re-runs the same loops with Kimi K2 Instruct in the prover role only.

Retrieval mode (prover input)	Sonnet prover	Kimi prover
No retrieval	4 (4.0%)	1 (1.0%)
LeanSearch v2 (standard mode)	14 (14.0%)	8 (8.0%)
LeanSearch v2 (reasoning mode)	20 (20.0%)	12 (12.0%)

The relative ordering of the three retrieval modes is preserved when the prover is downgraded from Sonnet to Kimi: reasoning mode (12) > standard mode (8) > no retrieval (1), so the marginal contribution of better retrieval information — both at round one (sketch and aggregated premises from reasoning mode) and at reflection (LeanSearch hits via standard mode) — carries through to a weaker prover backbone. The absolute success rates drop by roughly 40% ($14 \rightarrow 8$, $20 \rightarrow 12$) when Sonnet is replaced by Kimi, indicating that prover capability and retrieval capability contribute roughly multiplicatively to end-to-end proof success rather than acting as substitutes.

D.3 Reflection rounds in reasoning mode

The reasoning-mode loop in §3.2 caps revision at three rounds and uses the feasibility judge’s accept/reject decision to decide when the sketch is ready for downstream consumption. To isolate the contribution of the judge-driven reflection step, we re-run the Prove task on FATE-H with reasoning mode configured to use zero revisions: the initial sketch is generated as usual, but the judge’s verdict is ignored and the filtered premise set is passed straight to the prover regardless of whether the judge would have accepted it. Every other component of the loop (sketch generator, filter, prover, REPL verifier, hyperparameters) is held fixed at the production setting. Comparing the two runs measures the marginal contribution of the reflection step under a single-attempt budget.

Table 17: **Reflection-rounds ablation on FATE-H** ($n=100$). $rfl = 0$: the judge’s verdict is ignored and the initial sketch’s filtered premise set is passed straight to the prover. $rfl = 3$: production setting, the sketch reviser is invoked up to three times before the judge accepts or the loop falls back to the latest filtered output. Other reasoning-mode components are identical.

Reasoning-mode reflection	# solved	success rate
$rfl = 0$ (judge bypassed)	16	16.0%
$rfl = 3$ (production)	20	20.0%

The full reflection loop adds 4 solved problems on FATE-H over the no-reflection variant, a + 4 percentage-point gain that confirms the qualitative design argument of §3.2: the judge’s structured rejection signal lets the sketch reviser repair sketches whose initial premise set the prover cannot complete. The gain is consistent with our reading of Appendix C.1, where roughly one third of accepted queries needed at least one revision before the judge accepted, and is small enough on absolute terms that the ablation does not undermine the main-table claim — reasoning mode wins on FATE-H even when reflection is disabled, retaining a + 2-point lead over the strongest non-reasoning retriever (INF-X-Retriever at 16.0% in Table 3). Disabling reflection is therefore a meaningful but bounded loss; we keep it on by default because the additional cost is modest (Appendix C.1 reports a mean of one revision call per accepted query).

E Compute Resources and Licenses

E.1 Compute resources

GPU computation uses a small accelerator pool, and LLM-driven steps use the Claude Sonnet 4.5 and Kimi K2 Instruct hosted APIs. For standard mode (§4.1), embedding the entire Mathlib corpus with Qwen3-Embedding-8B runs on a single accelerator and completes within approximately six hours, with reranking performed on demand for the per-query top-50; in downstream experiments standard mode is exposed as a service hosted on two accelerators, returning a ranked list in roughly 0.2 seconds per query. For reasoning mode (§4.2), running the full sketch–filter–judge–reviser loop on a single query costs approximately \$1.10 USD at current API pricing. The remaining compute — running the reasoning-retrieval baselines (INF-X, ReasonIR, DIVER and its sub-configurations), the premise-selection baselines on extracted proof states, and the per-slice and ablation runs reported in the appendix — amounts to roughly three to four GPU-hours in total.

E.2 Licenses for existing assets

- **Mathlib** [The mathlib Community, 2020]: Apache 2.0 license. We use versions v4.28.0-rc1 and v4.29.1.
- **Jixia**: Apache 2.0 license. <https://github.com/frenzymath/jixia>.

- **Qwen3-32B** [Yang et al., 2025]: Apache 2.0 license.
- **Qwen3-Embedding-8B** and **Qwen3-Reranker-8B** [Zhang et al., 2025]: Apache 2.0 license.
- **Kimi K2 Instruct** [Team et al., 2025]: modified MIT license.
- **Claude Sonnet 4.5** [Anthropic, 2025]: accessed via the Anthropic API under its published usage policy.
- **Gemini 2.5 Pro** [Comanici et al., 2025]: accessed via the Google AI API under its published terms of service.
- **FATE-H** [Jiang et al., 2025]: MIT License.

All baseline retrieval systems (LeanExplore, LeanFinder, ReProver, LeanPremise, LeanStateSearch, INF-X-Retriever, ReasonIR, DIVER) are cited in the main text and accessed via their published code or hosted APIs.